PRD: Subscription Management System (Hackathon Prototype)

Scenario: Brightloop Software | **Scope:** one-day working prototype | **Owner:** Luffy

1. Goal

Build a working end-to-end subscription billing platform: products and plans, quotes to subscriptions, auto-generated invoices, payments through a real gateway (test mode), and a finance dashboard. On top of the manual's requirements, add multi-currency, autopay, membership analytics with a middle-tier upsell, notifications, Excel export, and expense / profit trackers.

Prototype rules: happy path must work live in the demo. Skip production concerns (HA, audit logs, full test coverage, advanced security hardening, PCI scope). Seed data is fine. Mock anything that would take more than 30 minutes.

2. Users and Roles

Role	Can do
Admin	Everything: products, plans, taxes, discounts, add staff, view all reports
Staff (internal)	Manage customers, subscriptions, invoices, record payments. Cannot create discounts/taxes or add staff
Customer (portal)	See own subscriptions, invoices, pay, toggle autopay, view own expense tracker, change currency

3. Scope: MoSCoW

Must have (demo cannot work without these)

1. Signup/login, unique email, password rule (more than 8 chars, upper, lower, special), JWT, role-based access, password reset link via email
2. Product CRUD with variants (e.g. Brand = Odoo adds 560 to price), sales price, cost price, notes

3. Plan CRUD: price, billing period (daily/weekly/monthly/yearly), min quantity, valid from/to, flags: auto-close, closable, pausable, renewable
4. Quotation template, then subscription (numbered, customer, plan, start/expiry, payment terms, order lines)
5. Subscription lifecycle: Draft, Quotation, Confirmed, Active, Closed
6. Invoice generation from subscription (lines, tax, totals auto-filled), lifecycle Draft, Confirmed, Paid, plus cancel/send/print
7. Billing job: scheduled run creating invoices for due subscriptions, idempotent (no double bill, none skipped)
8. Tax rules (percentage, applied automatically) and discount rules (fixed/percent, min amount/qty, dates, usage limit, per product or whole subscription, auto-applied on next invoice)
9. Payment recording (method, amount, date, note) against an invoice, plus gateway checkout
10. Finance dashboard: active subs, revenue, outstanding, overdue invoices, filters

Should have (your extras, all in prototype form)

11. Multi-currency
12. Autopay
13. Notifications
14. Membership metrics and middle-tier upsell nudge
15. Excel export
16. Customer expense tracker
17. Profit / loss / most profitable subscriptions tracker

Won't do (explicitly out of scope)

Real bank reconciliation, dunning email sequences, proration math beyond simple, multi-tenant, refunds workflow, PDF invoice design polish (basic print view is enough).

4. Functional Requirements

4.1 Auth and Roles

- Register with email + password. Reject duplicate email. Validate password rule on both client and server.
- Forgot password: generate a one-time token (expires in 30 min), email a reset link (use Mailtrap or Gmail SMTP for the demo).

- Only Admin can create Staff accounts. Customer self-signup always creates the Customer role.
- Route guards by role (frontend) plus `@PreAuthorize` style checks (backend).

4.2 Products, Variants, Plans

- **Product:** name, type, sales price, cost price, notes.
- **Variant:** attribute (e.g. Brand), value (e.g. Odoo), extra price (+560). Final line price = product price + variant extra.
- **Plan:** name, price, billing period, min quantity, available-from/to, and four admin switches: auto-close at end, can close early, can pause, can renew.
- Plans carry a **tier** field (Low / Medium / High) used by the upsell feature.

4.3 Quotation Template to Subscription

- Template: name, validity in days, plan, product lines.
- "Create subscription from template" copies lines and sets the quote validity.
- Subscription fields: auto number (SUB-0001), customer, plan, start date, expiry date, payment terms, lines (product, qty, unit price, tax, amount).
- Status transitions are button-driven and visible as a status bar on the detail page: Draft, Quotation, Confirmed, Active, Closed.
- Pause/renew/close actions only appear if the plan's switches allow them.

4.4 Invoicing

- Invoice is created from a subscription by copying customer, lines, tax and totals.
- **Billing job** (Spring `@Scheduled`, plus a manual "Run billing now" admin button for the demo): finds Active subscriptions where `next_billing_date <= today`, creates one invoice, advances `next_billing_date`.
- **No double-billing:** unique constraint on `(subscription_id, billing_period_start)`. **No skipping:** the job loops until `next_billing_date > today` (catches missed days).
- Tax computed from admin tax rules (percentage per product or default). Active discounts are applied automatically when the invoice is built, and usage count is incremented.
- Invoice actions: Confirm, Cancel, Send (email), Print (browser print stylesheet). Status: Draft, Confirmed, Paid.

4.4a GST and Place of Supply (India)

Invoices split GST into CGST + SGST for same-state customers and charge IGST for other-state customers, decided by one hardcoded place-of-supply rule in the invoice generator.

Company: registered in Karnataka (mock GSTIN, stored in config). **Supported customer states:** Karnataka, Tamil Nadu, Kerala, and North (one bucket standing for any north-Indian state, e.g. Delhi, treated as a single other-state value).

Customer state	Company state	Tax charged (18% example)
Karnataka	Karnataka	CGST 9% + SGST 9%
Tamil Nadu	Karnataka	IGST 18%
Kerala	Karnataka	IGST 18%
North	Karnataka	IGST 18%

- Rule: `customer.state == company.state` then split the tax rate evenly into CGST and SGST; otherwise apply the full rate as IGST.
- UTGST is kept in the `gst_type` enum but unused in the prototype.
- Customers billed in a foreign currency are treated as exports: no GST line (assumption, easy to switch off).
- Calculation on invoice creation, per line:

```
\text{Line Net} = (\text{Price} \times \text{Qty}) - \text{Discount}
\text{Tax} = \text{Line Net} \times \frac{\text{Tax Percent}}{100}
```

- **Print view:** show the company GSTIN and the customer GSTIN (for B2B) in the header, the HSN/SAC code beside each product or variant line, and a breakdown under the subtotal with separate CGST, SGST and IGST rows instead of one "Tax" row.

4.5 Discounts

- Admin-only. Type fixed or percent, min purchase amount or min quantity, start/end date, max usage count.
- Applies to specific products or whole subscription.
- Invoice generator checks validity (date window, usage < limit, min condition) and attaches the discount line automatically.

4.6 Payments and Gateway

- **Gateway:** Razorpay test mode (best fit for India, supports INR and international currencies in test) or Stripe test mode as the fallback. Pick one, do not build both.
- Flow: customer clicks Pay on an invoice, backend creates a gateway order, frontend opens checkout, gateway webhook (or client success callback for the prototype) marks the payment, creating a Payment record and flipping the invoice to Paid.
- Manual payment recording for staff: method (cash, bank transfer, UPI, card, cheque), amount, date, note, linked invoice.
- Partial payments: allowed, invoice becomes Paid when sum of payments >= total.
- Overdue = Confirmed invoice past due date (due date = invoice date + payment terms).

4.7 Multi-Currency (prototype version)

- Each customer has a preferred currency. Base currency of the company is INR.
- A `currency_rates` table holds rates against INR (seed USD, EUR, GBP, AED, SGD). Admin can edit rates manually; optional: nightly fetch from a free rates API.
- Prices are stored in base currency. Display and invoices convert at invoice-creation time, and the **rate is frozen on the invoice** (store `currency`, `exchange_rate`, `total_in_base`) so history never shifts.
- Dashboard totals always report in base currency, with a currency switcher for display.
- Gateway charge uses the invoice currency.

4.8 Autopay

- Customer toggles autopay and picks a saved payment method (gateway customer + token, or Razorpay subscription/mandate in test mode).
- When the billing job confirms an invoice for an autopay customer, it immediately attempts a charge.
- On success: payment recorded, invoice Paid, notification sent. On failure: invoice stays Confirmed, customer notified, retry once next day (simple `retry_count` field, max 2).
- **Prototype shortcut:** if tokenized charging is slow to wire, simulate with a "stored test card" flag and call the gateway's test charge. Be upfront in the demo about what is simulated.

4.9 Notifications

Event-driven, written to a `notifications` table and shown in a bell icon with unread count. Email is sent for the important ones.

Event	In-app	Email
Invoice created	Yes	Yes
Payment received / autopay success	Yes	Yes
Autopay failed	Yes	Yes
Invoice overdue	Yes	Yes
Subscription expiring in 7 days	Yes	Yes
Plan or discount created (staff)	Yes	No

The scheduler produces the overdue and expiry reminders once a day. Browser web-push is a stretch goal only.

4.10 Membership Metrics and Middle-Tier Upsell

Metrics page (Admin/Staff):

- Top plans: ranked by active subscriber count and by revenue.
- Underperforming plans: lowest subscriber count and highest churn (closed / total), flagged with a red badge below a configurable threshold.
- Tier distribution chart: Low vs Medium vs High share.
- Conversion funnel: Quotation to Confirmed to Active.

Pricing page with middle-tier nudge (Customer):

- Show three cards side by side: Low, Medium (highlighted "Most Popular"), High.
- Under the Medium card show a computed difference message, for example: *"Only ₹X more per month for High: you get [feature list] that Medium doesn't include."* Under the High card show the per-feature value gain.
- The Medium plan is priced so the gap to High is small relative to the extra features (a standard decoy/anchoring layout). The message text is generated from real plan data (price delta, feature delta), not hardcoded.
- Log which card was viewed and chosen so the metrics page can show whether the nudge improves High-tier conversion.
- Keep the claims accurate: only list features that actually differ. This is persuasion by clear comparison, not misleading the customer.

4.11 Excel Export

- "Export to Excel" button on: invoices, payments, subscriptions, and the profit report. Apache POI generates `.xlsx`, honoring the active filters.
- Include a summary sheet (totals) plus a data sheet.
- **GST Report sheet:** one row per invoice line with invoice number, date, customer, customer state, GSTIN, HSN/SAC, taxable value, CGST, SGST, IGST and total, so the export doubles as a mock GSTR-1 filing helper.

4.12 Customer Expense Tracker (portal)

- Customer sees all their subscription charges: per subscription, per month, total spend, upcoming renewals (next 30 days), and spend by currency.
- Simple bar chart (monthly spend) and a pie (spend by subscription).
- Optional: customer sets a monthly budget and sees a warning when projected spend exceeds it.

4.13 Profit / Loss Tracker (Admin/Finance)

- Per subscription: revenue (paid invoices) minus cost (sum of product cost price x quantity per billing cycle) = gross margin and margin %.
- Rolled up per plan, per product, and per month.
- "Most profitable" and "loss-making" leaderboards (top 5 / bottom 5).
- Summary cards: total revenue, total cost, gross profit, margin %, MRR (monthly recurring revenue, normalising yearly plans by /12 and weekly by x4.33), outstanding amount.
- **GST is not revenue:** margin, revenue, MRR and leaderboards use the invoice subtotal (after discount, before GST). CGST, SGST and IGST are a pass-through liability and shown separately as "GST collected" on the profit page.

4.14 Finance Dashboard

- Cards: active subscriptions, revenue (period), payments received, outstanding, overdue count.
- Table of overdue invoices with days overdue and a "Send reminder" action.
- Filters: date range, customer, plan, status, currency.
- All list pages paginated server-side, with DB indexes on `status`, `next_billing_date`, `customer_id`, `due_date`, to stay under 2 seconds on seeded thousands of rows.

5. Data Model (core tables)

- `users` (id, email unique, password_hash, role, preferred_currency, autopay_enabled, gateway_customer_id)
- `products` (id, name, type, sales_price, cost_price, notes)
- `product_variants` (id, product_id, attribute, value, extra_price)
- `plans` (id, name, tier, price, billing_period, min_qty, available_from, available_to, auto_close, closable, pausable, renewable, features JSON)
- `quotation_templates` (id, name, validity_days, plan_id) and `template_lines`
- `subscriptions` (id, number, customer_id, plan_id, start_date, expiry_date, payment_terms_days, status, next_billing_date)
- `subscription_lines` (id, subscription_id, product_id, variant_id, qty, unit_price, tax_id, amount)
- `taxes` (id, name, percent) and `discounts` (id, type, value, min_amount, min_qty, start, end, max_uses, used_count, applies_to, product_id)
- `invoices` (id, number, subscription_id, billing_period_start, customer_id, status, currency, exchange_rate, subtotal, tax_total, discount_total, total, due_date, retry_count) with unique `(subscription_id, billing_period_start)`
- `invoice_lines`
- `payments` (id, invoice_id, method, amount, paid_on, note, gateway_ref)
- `currency_rates` (currency, rate_to_inr, updated_at)
- `notifications` (id, user_id, type, message, read, created_at)
- `plan_views` (id, user_id, plan_id, action viewed/selected, created_at) for nudge analytics
- `password_reset_tokens`

GST additions to the tables above

- `taxes`: add `gst_type` (CGST, SGST, IGST, UTGST) and `hsn_sac_code` (VARCHAR, needed for Indian B2B invoicing)
- `users` (customers): add `state` (Karnataka, Tamil Nadu, Kerala, North) and optional `gstin`
- `subscription_lines` and `invoice_lines`: add `hsn_sac_code`, `cgst_amount`, `sgst_amount`, `igst_amount`
- `invoices`: add `company_gstin`, `customer_gstin`, `place_of_supply`

6. Suggested Stack (matches your Java track)

- **Backend:** Java + Spring Boot (Web, Data JPA, Security + JWT, Scheduling, Mail), Apache POI for Excel
- **DB:** PostgreSQL (or H2 if setup time is tight, swap later)
- **Frontend:** React + Vite + Tailwind, Recharts for charts
- **Gateway:** Razorpay test mode SDK
- **Email:** Mailtrap sandbox SMTP
- **Deploy:** run locally for the demo, optional Render/Railway

7. One-Day Build Plan

Block	Hours	Output
Setup and schema	1	Spring Boot + DB + entities + seed script (users, 3 tiers of plans, 2,000+ fake subscriptions)
Auth and roles	1	Signup, login, reset email, guards
Catalog: products, variants, plans, taxes, discounts	1.5	CRUD APIs and admin screens
Templates, subscriptions, lifecycle	1.5	Quote to Active flow with status bar
Billing job and invoices	1.5	Idempotent generator, tax and discount application, actions
Payments and gateway	1.5	Razorpay checkout, manual payments, paid/overdue logic
Dashboard and profit tracker	1.5	Cards, tables, filters, margin leaderboards
Extras: currency, autopay, notifications, upsell, Excel, expense tracker	3	In that order of priority; cut from the bottom if late
Polish and demo prep	1	Seed demo data, rehearse script

Cut order if time runs out: web-push, budget alerts, live FX fetch, retry logic, then simulate autopay. Never cut the Must-haves.

8. Acceptance Checklist (demo script)

1. Sign up with a weak password: rejected. Sign up with a valid one: success. Duplicate email: rejected.
2. Admin creates a product with a +560 variant, a plan, a tax and a 10% discount.
3. Staff creates a subscription from a template, moves it Draft to Active.
4. Click "Run billing now": invoice appears with tax and discount applied. Click again: no duplicate.
5. Customer logs in, switches currency to USD, pays via Razorpay test card: invoice becomes Paid, notification arrives.
6. Customer enables autopay: next billing run charges automatically.
7. Pricing page shows Low / Medium / High with the difference message.
8. Dashboard shows active subs, revenue, outstanding, overdue; filters work; pages load in under 2 seconds.
9. Metrics page shows top and underperforming plans; profit page shows most profitable and loss-making subscriptions.
10. Export to Excel downloads a correct file.
11. GST check: a Karnataka customer's invoice shows CGST 9% + SGST 9%; a Tamil Nadu, Kerala or North customer's shows IGST 18%; the print view shows GSTINs and HSN/SAC; the Excel GST sheet matches.

9. Risks and Mitigations

- **Gateway integration eats the day:** wire it by hour 6, with manual payment as the fallback.
- **Autopay tokenization complexity:** simulate with stored test credentials and say so.
- **Currency drift:** freeze the rate on each invoice.
- **Performance with large data:** server-side pagination and indexes from the start, seed 5,000 rows early to catch slow queries.
- **Scope creep:** the Must list is the contract; extras are built only after the Must flow works end to end.

10. Success Metrics (for judges)

- Full quote-to-cash flow works live without manual DB edits
- Every scene from the manual is demonstrable
- Each extra (currency, autopay, upsell, notifications, Excel, trackers) shows at least one working screen